Квантизация LLM



Карим Крамин

План доклада

1. Мотивация и базовые понятия

Зачем квантизация, память GPU, формула сетки

2. GPTQ

Послойная задача, Гессиан, три ускорения

3. SpinQuant

Повороты, R_1-R_4, Cayley SGD

4. Scaling Laws for Precision

Законы масштабирования и точность

5. KVQuant

KV-кэш для длинного контекста

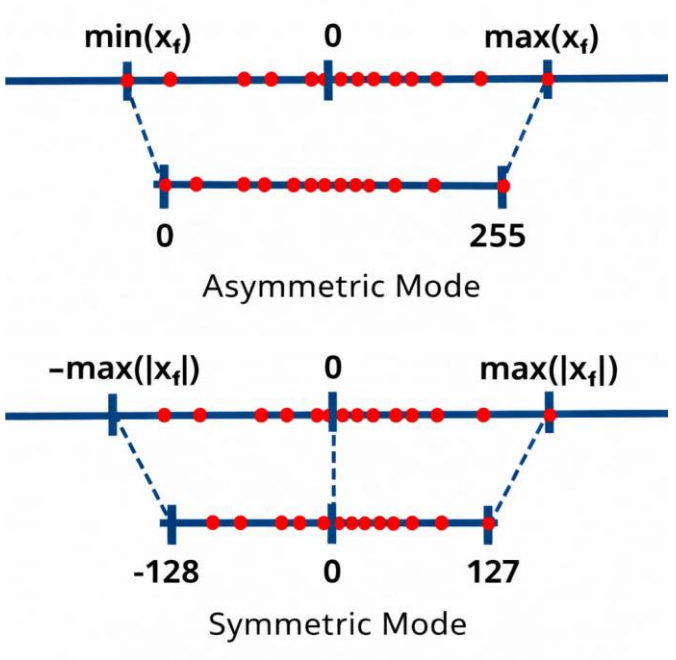
6. Сравнение и выводы

Что такое квантизация

$$X_Q = \alpha \lfloor \frac{X_R - \beta}{\alpha} \rceil + \beta$$

где $\alpha = \frac{\max(X_R) - \min(X_R)}{2^N - 1}, \beta = \min(X_R)$
в не симметричном квантизации

где $\alpha = \frac{\max(|X_R|)}{2^{N-1} - 1}, \beta = 0$
в симметричном квантизации



Обозначение: W4A8KV8 = веса 4 бит, активации 8 бит, KV-кэш 8 бит

Зачем нужна квантизация

GPT-3 175B:
FP16: 326 ГБ
INT4: 82 ГБ
INT3: 63 ГБ

A100 GPU = 80 ГБ

HBM (видеопамять): 80 ГБ
Регистры: 27 МБ

Вычисления производятся только в регистрах, из-за медленной скорости загрузки данных из VRAM GPU может простаивать

GPU	FP16	3bit	Speedup	GPU reduction
A6000 – 48GB	589ms	130ms	4.53×	8 → 2
A100 – 80GB	230ms	71ms	3.24×	5 → 1

Table 6: Average per-token latency (batch size 1) when generating sequences of length 128.

GPTQ

Accurate Post-Training Quantization
for Generative Pre-Trained Transformers

ICLR 2023

175B

Квантизирует параметры
за +-4 часа

x4.5

ускорение
инференса



GPTQ: послойная задача и Гессиан

$$\text{argmin}_{\widehat{\mathbf{W}}} ||\mathbf{WX} - \widehat{\mathbf{W}}\mathbf{X}||_2^2.$$

$$\arg\min_{\hat{w}} \|wX - \hat{w}X\|^2 =$$

$$= \arg\min_{\delta} \|\delta X\|^2 = \arg\min_{\delta} \delta^T X X^T \delta$$

RTN (Round-to-Nearest):
ошибки накапливаются.

GPTQ:
При квантизации w_q корректируем все остальные веса через обратный гессиан H^{-1} для минимизации ошибки.

2 Optimal Brain Surgeon

In deriving our method we begin, as do Le Cun, Denker and Solla [1990], by considering a network trained to a local minimum in error. The functional Taylor series of the error with respect to weights (or parameters, see below) is:

$$\delta E = \left(\frac{\partial E}{\partial \mathbf{w}}\right)^T \cdot \delta\mathbf{w} + \frac{1}{2}\delta\mathbf{w}^T \cdot \mathbf{H} \cdot \delta\mathbf{w} + O(\|\delta\mathbf{w}\|^3) \quad (1)$$

where $\mathbf{H} \equiv \partial^2 E/\partial\mathbf{w}^2$ is the Hessian matrix (containing all second order derivatives) and the superscript T denotes vector transpose. For a network trained to a local minimum in error, the first (linear) term vanishes; we also ignore the third and all higher order terms. Our goal is then to set one of the weights to zero (which we call w_q) to minimize the increase in error given by Eq. 1. Eliminating w_q is expressed as:

$$\delta w_q + w_q = 0 \qquad \text{or more generally} \quad \mathbf{e}_q^T \cdot \delta\mathbf{w} + w_q = 0 \quad (2)$$

where $\mathbf{e}_q$ is the unit vector in weight space corresponding to (scalar) weight w_q. Our goal is then to solve:

$$Min_q\{Min_{\delta\mathbf{w}}\{\tfrac{1}{2}\delta\mathbf{w}^T \cdot \mathbf{H} \cdot \delta\mathbf{w}\} \quad \text{such that} \quad \mathbf{e}_q^T \cdot \delta\mathbf{w} + w_q = 0\} \quad (3)$$

To solve Eq. 3 we form a Lagrangian from Eqs. 1 and 2:

$$L = \tfrac{1}{2}\delta\mathbf{w}^T \cdot \mathbf{H} \cdot \delta\mathbf{w} + \lambda(\mathbf{e}_q^T \cdot \delta\mathbf{w} + w_q) \quad (4)$$

where λ is a Lagrange undetermined multiplier. We take functional derivatives, employ the constraints of Eq. 2, and use matrix inversion to find that the optimal weight change and resulting change in error are:

$$\delta\mathbf{w} = -\frac{w_q}{[\mathbf{H}^{-1}]_{qq}}\mathbf{H}^{-1} \cdot \mathbf{e}_q \qquad \textit{and} \qquad L_q = \frac{1}{2}\frac{w_q^2}{[\mathbf{H}^{-1}]_{qq}} \quad (5)$$

Note that neither $\mathbf{H}$ nor $\mathbf{H}^{-1}$ need be diagonal (as is assumed by Le Cun et al.); moreover, our method recalculates the magnitude of *all* the weights in the network, by the left side of Eq. 5. We call L_q the "saliency" of weight q — the increase in error that results when the weight is eliminated — a definition more general than Le Cun et al.'s, and which includes theirs in the special case of diagonal $\mathbf{H}$.

Thus we have the following algorithm:

Optimal Brain Surgeon procedure

1. Train a "reasonably large" network to minimum error.
2. Compute $\mathbf{H}^{-1}$.
3. Find the q that gives the smallest saliency $L_q = w_q^2/(2[H^{-1}]_{qq})$. If this candidate error increase is much smaller than E, then the qth weight should be deleted, and we proceed to step 4; otherwise go to step 5. (Other stopping criteria can be used too.)
4. Use the q from step 3 to update *all* weights (Eq. 5). Go to step 2.
5. No more weights can be deleted without large increase in E. (At this point it may be desirable to retrain the network.)

GPTQ: послойная задача и Гессиан

$$\text{argmin}_{\widehat{\mathbf{W}}} ||\mathbf{WX} - \widehat{\mathbf{W}}\mathbf{X}||_2^2. \qquad \text{Hessian is } \mathbf{H}_F = 2\mathbf{X}_F\mathbf{X}_F^\top$$

$$w_q = \text{argmin}_{w_q} \frac{(\text{quant}(w_q) - w_q)^2}{[\mathbf{H}_F^{-1}]_{qq}}, \qquad \boldsymbol{\delta}_F = -\frac{w_q - \text{quant}(w_q)}{[\mathbf{H}_F^{-1}]_{qq}} \cdot (\mathbf{H}_F^{-1})_{:,q}.$$

$$e_q = \hat{w}_q - w_q \qquad \delta_j = -e_q \frac{(H^{-1})_{jq}}{(H^{-1})_{qq}}$$

Если j и q коррелируют то H^-1 > 0

Гессиан H = XX^T

H[ii] — влияние веса i на выход нейрона
Большой - вес важен, влияет сильно
H[ij] — корреляция весов i и j
Ненулевой - можно компенсировать ошибку
$H^{-1}[qq]$ — чувствительность ошибки к изменению w_q.
$\Delta L = (w_q - \text{quant}(w_q))^2 / 2\, H^{-1}[qq]$

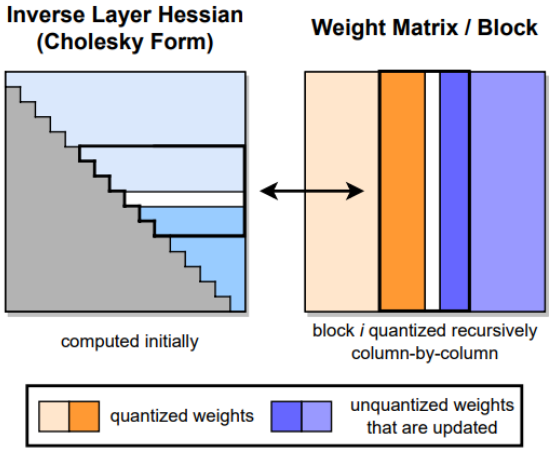
$$\mathbf{H}_{-q}^{-1} = \left(\mathbf{H}^{-1} - \frac{1}{[\mathbf{H}^{-1}]_{qq}} \mathbf{H}_{:,q}^{-1} \mathbf{H}_{q,:}^{-1}\right)_{-p}.$$

GPTQ: алгоритм и три ускорения

1. Фиксированный порядок
Все строки, один порядок, H^{-1} общий

2. Пакетные обновления (B=128)
Блоки колонок, матричные умножения

3. Разложение Холецкого
Предвычисление H^{-1} и нет накопления ошибок



The Full Algorithm. Finally, we present the full pseudocode for GPTQ in Algorithm 1, including the optimizations discussed above.

Algorithm 1 Quantize $\mathbf{W}$ given inverse Hessian $\mathbf{H}^{-1} = (2\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I})^{-1}$ and blocksize B.

$\mathbf{Q} \leftarrow \mathbf{0}_{d_{\text{row}} \times d_{\text{col}}}$ *// quantized output*
$\mathbf{E} \leftarrow \mathbf{0}_{d_{\text{row}} \times B}$ *// block quantization errors*
$\mathbf{H}^{-1} \leftarrow \text{Cholesky}(\mathbf{H}^{-1})^\top$ *// Hessian inverse information*
for $i = 0, B, 2B, \ldots$ **do**
  for $j = i, \ldots, i + B - 1$ **do**
    $\mathbf{Q}_{:,j} \leftarrow \text{quant}(\mathbf{W}_{:,j})$ *// quantize column*
    $\mathbf{E}_{:,j-i} \leftarrow (\mathbf{W}_{:,j} - \mathbf{Q}_{:,j}) \,/\, [\mathbf{H}^{-1}]_{jj}$ *// quantization error*
    $\mathbf{W}_{:,j:(i+B)} \leftarrow \mathbf{W}_{:,j:(i+B)} - \mathbf{E}_{:,j-i} \cdot \mathbf{H}^{-1}_{j,j:(i+B)}$ *// update weights in block*
  end for
  $\mathbf{W}_{:,(i+B):} \leftarrow \mathbf{W}_{:,(i+B):} - \mathbf{E} \cdot \mathbf{H}^{-1}_{i:(i+B),(i+B):}$ *// update all remaining weights*
end for

Результаты

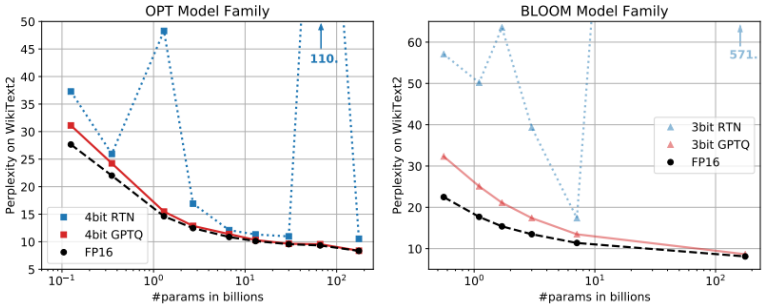


Figure 1: Quantizing OPT models to 4 and BLOOM models to 3 bit precision, comparing GPTQ with the FP16 baseline and round-to-nearest (RTN) (Yao et al., 2022; Dettmers et al., 2022).

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B	175B
full	16	27.65	22.00	14.63	12.47	10.86	10.13	9.56	9.34	8.34
RTN	4	37.28	25.94	48.17	16.92	12.10	11.32	10.98	110	10.54
GPTQ	4	**31.12**	**24.24**	**15.47**	**12.87**	**11.39**	**10.31**	**9.63**	**9.55**	**8.37**
RTN	3	1.3e3	64.57	1.3e4	1.6e4	5.8e3	3.4e3	1.6e3	6.1e3	7.3e3
GPTQ	3	**53.85**	**33.79**	**20.97**	**16.88**	**14.86**	**11.61**	**10.27**	**14.16**	**8.68**

Table 3: OPT perplexity results on WikiText2.

Результаты

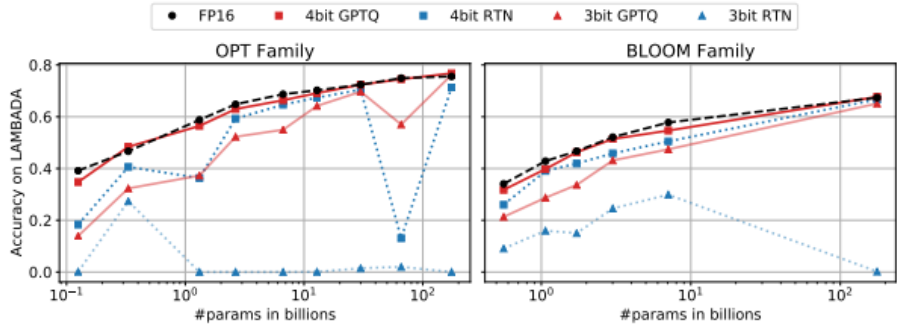


Figure 3: The accuracy of OPT and BLOOM models post-GPTQ, measured on LAMBADA.

Method	Bits	OPT-175B Wiki2	PTB	C4	LAMB. ↑	BLOOM-176B Wiki2	PTB	C4	LAMB. ↑
Baseline	16	8.34	12.01	10.13	75.59	8.11	14.59	11.71	67.40
RTN	4	10.54	14.22	11.61	71.34	8.37	15.00	12.04	66.70
GPTQ	4	**8.37**	**12.26**	**10.28**	**76.80**	**8.21**	**14.75**	**11.81**	**67.71**
RTN	3	7.3e3	8.0e3	4.6e3	0	571.	107.	598.	0.17
GPTQ	3	**8.68**	**12.68**	**10.67**	**76.19**	**8.64**	**15.57**	**12.27**	**65.10**
GPTQ	3/g1024	8.45	12.48	10.47	77.39	8.35	15.01	11.98	67.47
GPTQ	3/g128	8.45	12.37	10.36	76.42	8.26	14.89	11.85	67.86

Table 5: Results summary for OPT-175B and BLOOM-176B. “g1024” and “g128” denote results with groupings of size 1024 and 128, respectively.

GPTQ: ограничения

1. Квантизация только весов
Мы получили: **W4A16KV16**
Активации и KV-кэш остаются в FP16

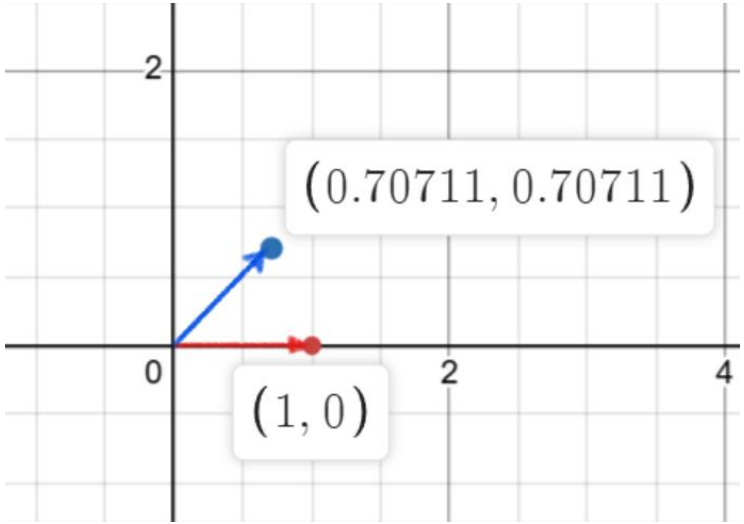
2. Проблема выбросов остаётся
Некоторые каналы имеют значения на порядок больше остальных:
Итого: сетка квантования растягивается и большинство каналов теряет точность



SpinQuant

LLM Quantization with Learned Rotations

ICLR 2025



Зачем повороты

Поворот: $x' = Rx$

$$R^T R = I$$

Инвариантность выхода:

$$y = Wx$$

$$y = WR^T Rx$$

$$y = (WR^T)(Rx)$$

$$W' = WR^T, \quad x' = Rx$$

$$y = W'x'$$

Квантизация:

$$y \approx Q(W)\, Q(x)$$

$$y \approx Q(WR^T)\, Q(Rx)$$

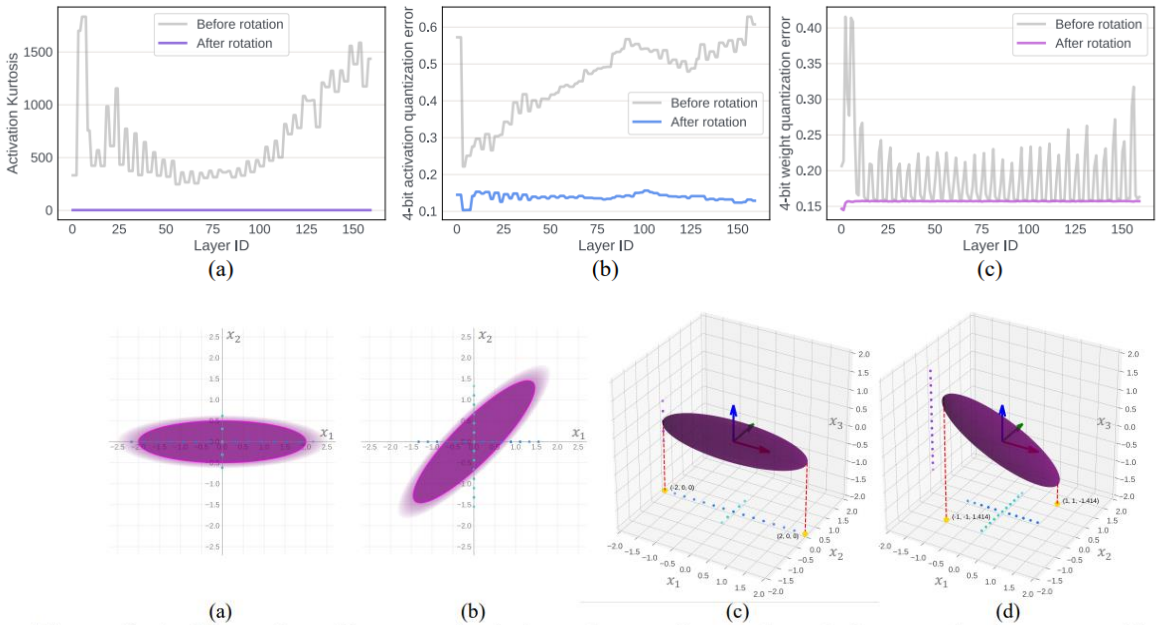
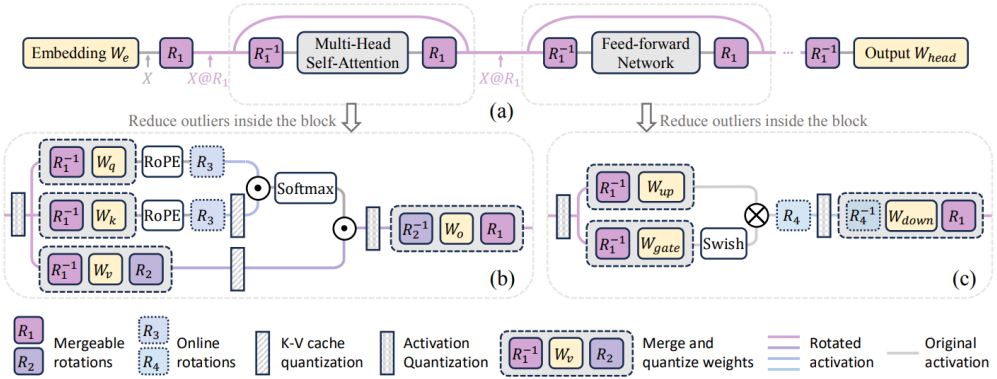


Figure 6: An illustration of how rotation helps reduce outliers and maximize quantization range utilization.

Четыре матрицы поворота



R₁, R₂ - заранее в весах

$$Q = W_Q E t$$

$$Q = (W_Q R_1^T)(R_1 E) t$$

Без дополнительных вычислительных затрат при инференсе.

R₃, R₄ - Hadamard матрицы

$$Q = R_3 \,\mathrm{RoPE}\, W'_Q x'$$

$$K = R_3 \,\mathrm{RoPE}\, W'_K x'$$

$$Q^T K$$

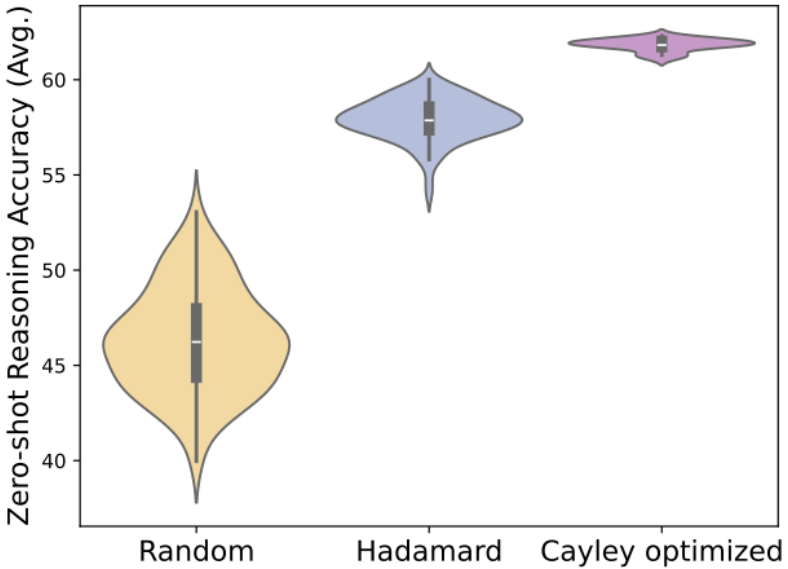
Быстрое умножение Hadamard: O(n log n)
+8% вычислений, потому что повороты придется делать в процессе инференса.

Оптимизация поворотов

Cayley SGD

Оптимизация R_1,R_2 на многообразии Штифеля (множество ортогональных матриц)

R·R^T = I после каждого шага



SpinQuant результаты

Table 1: Comparison of the perplexity score on WikiText2 and averaged accuracy on eight Zero-shot Common Sense Reasoning tasks. Results for SmoothQuant (Xiao et al., 2022), LLM-QAT (Liu et al., 2023c), GPTQ (Frantar et al., 2022) were obtained using their publicly released codebase. While OmniQuant (Shao et al., 2023), AWQ (Lin et al., 2023), and QuIP# (Tseng et al., 2024) results were quoted from their papers. Full results are in the Appendix.

#Bits (W-A-KV)	Method	LLaMA-2 7B 0-shot[8] Avg.(↑)	LLaMA-2 7B Wiki (↓)	LLaMA-2 13B 0-shot[8] Avg.(↑)	LLaMA-2 13B Wiki (↓)	LLaMA-2 70B 0-shot[8] Avg.(↑)	LLaMA-2 70B Wiki (↓)	LLaMA-3.2 1B 0-shot[8] Avg.(↑)	LLaMA-3.2 1B Wiki (↓)	LLaMA-3.2 3B 0-shot[8] Avg.(↑)	LLaMA-3.2 3B Wiki (↓)	LLaMA-3 8B 0-shot[8] Avg.(↑)	LLaMA-3 8B Wiki (↓)	Mistral-7B 0-shot[8] Avg.(↑)	Mistral-7B Wiki (↓)
16-16-16	FloatingPoint	66.9	5.5	68.3	5.0	72.9	3.3	56.9	13.4	63.9	10.7	69.6	6.1	71.0	5.4
4-8-16	RTN	62.4	7.9	57.3	6.7	68.6	5.0	55.4	20.7	58.6	29.0	65.5	8.2	59.3	6.8
	SmoothQuant	58.9	7.5	63.6	6.1	70.6	4.1	47.1	1e2	55.6	3e2	61.0	10.7	–	–
	LLM-QAT	64.8	11.4	67.5	14.5	–	–	53.2	21.0	60.8	41.1	67.2	7.7	–	–
	AWQ (w4)	–	6.2	–	5.1	–	–	–	–	–	–	–	–	–	–
	OmniQuant (w4)	–	5.7	–	5.0	–	3.5	–	–	–	–	–	–	–	–
	QuIP# (w4)	–	**5.6**	–	5.0	–	**3.4**	–	–	–	–	–	–	–	–
	GPTQ	64.9	20.2	65.2	5.9	71.7	4.3	55.0	17.3	58.7	25.2	64.5	7.2	51.7	8.6
	SpinQuant$_{no\ had}$	**65.7**	5.8	**68.2**	5.1	72.1	3.7	56.0	15.3	61.4	11.6	**68.6**	6.7	68.8	5.7
	SpinQuant$_{had}$	**65.7**	5.7	68.1	**5.0**	**72.7**	3.5	**56.5**	**14.4**	**63.2**	**11.5**	68.4	**6.5**	**69.9**	**5.5**
4-8-8	RTN	62.5	7.9	57.6	6.7	68.4	5.0	55.7	20.7	58.4	28.8	65.3	8.2	58.9	6.7
	SmoothQuant	58.8	7.5	63.4	6.1	70.5	4.1	47.1	1e2	55.5	3e2	60.9	10.7	–	–
	LLM-QAT	64.6	11.4	67.5	14.2	–	–	53.1	21.0	60.5	39.3	66.9	7.6	–	–
	GPTQ	64.8	20.2	65.3	5.9	71.6	4.3	54.8	17.3	58.7	24.1	64.6	7.2	51.7	8.6
	SpinQuant$_{no\ had}$	**65.8**	5.8	68.1	**5.1**	72.2	3.7	55.7	15.3	61.8	11.7	68.6	6.7	69.4	5.7
	SpinQuant$_{had}$	**65.8**	**5.7**	**68.2**	**5.1**	**72.7**	**3.5**	**55.8**	**14.3**	**63.2**	**11.2**	**68.8**	**6.5**	**70.2**	**5.5**
4-4-16	RTN	35.6	2e3	35.3	7e3	35.1	2e5	41.2	1e2	42.1	7e2	43.9	2e2	41.4	4e2
	SmoothQuant	41.8	3e2	44.9	34.5	57.7	57.1	37.9	2e3	43.6	4e2	40.3	9e2	–	–
	LLM-QAT	47.8	12.9	34.3	4e3	–	–	42.0	62.1	46.9	37.6	44.9	42.9	–	–
	GPTQ	36.8	9e3	35.2	5e3	35.5	2e6	41.6	1e2	43.4	3e2	40.6	2e2	40.4	3e2
	SpinQuant$_{no\ had}$	57.0	9.2	61.8	7.2	61.0	7.3	44.8	48.4	52.9	22.4	51.9	18.6	52.7	13.4
	SpinQuant$_{had}$	**64.1**	**5.9**	**67.2**	**5.2**	**71.0**	**3.8**	**53.5**	**15.3**	**61.0**	**11.1**	**65.8**	**7.1**	**68.4**	**5.7**
4-4-4	RTN	37.1	2e3	35.5	7e3	35.0	2e5	40.6	2e2	41.2	8e2	43.1	3e2	41.4	4e2
	SmoothQuant	39.0	7e2	40.5	56.6	55.9	10.5	36.5	2e3	40.0	6e2	38.7	2e3	–	–
	LLM-QAT	44.9	14.9	35.0	4e3	–	–	41.5	76.2	45.9	42.0	43.2	52.5	–	–
	GPTQ	36.8	9e3	35.2	5e3	35.6	1e6	41.6	1e2	41.1	4e2	40.5	2e2	41.3	2e2
	SpinQuant$_{no\ had}$	56.0	9.2	60.7	7.1	62.0	7.4	45.3	47.7	52.9	22.4	52.6	18.6	52.4	13.7
	SpinQuant$_{had}$	**64.0**	**5.9**	**66.9**	**5.3**	**71.2**	**3.8**	**53.4**	**15.9**	**60.5**	**11.4**	**65.5**	**7.3**	**68.6**	**5.8**

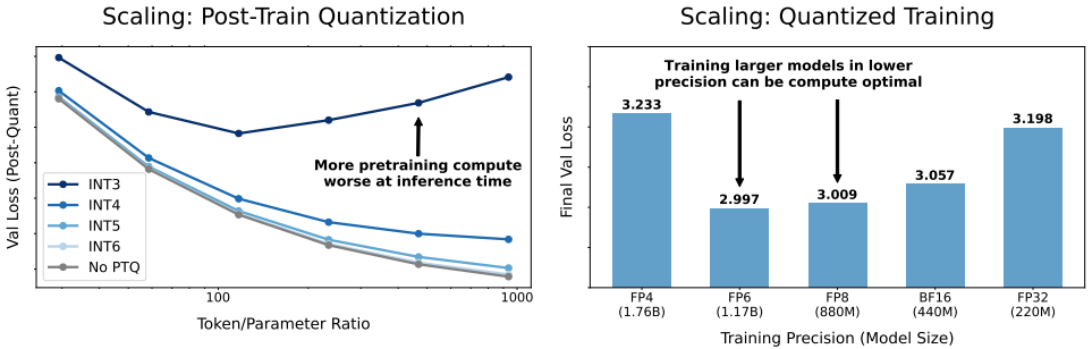
Scaling Laws for Precision

2024

465 предобученных моделей, 3-16 бит

Главная формула

$$L(N, D, P_{\text{train}}, P_{\text{post}}) = \underbrace{\underbrace{\underbrace{AN_{\text{eff}}^{-\alpha}}_{\text{Training-time Effects}} + BD^{-\beta} + E}_{\text{Usual Chinchilla form}} + \underbrace{\delta_{\text{PTQ}}(N_{\text{eff}}, D, P_{\text{train}}, P_{\text{post}})}_{\text{Post-Training Effects}}}$$

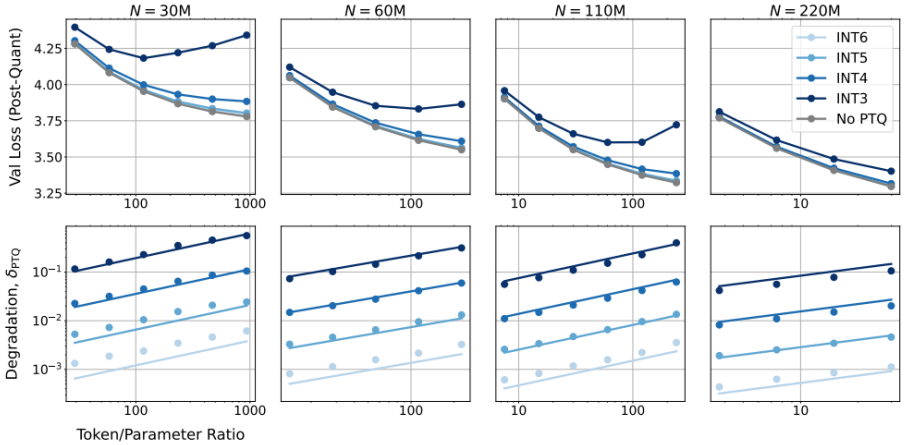


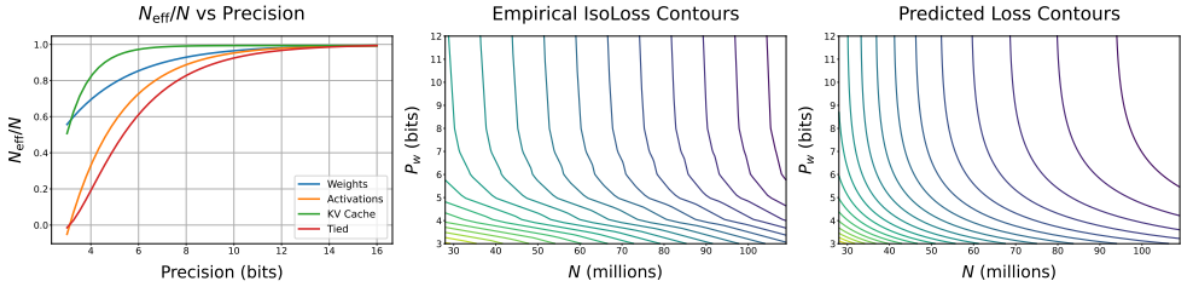
Выводы

Вывод 1: Переобученные модели деградируют больше при PTQ

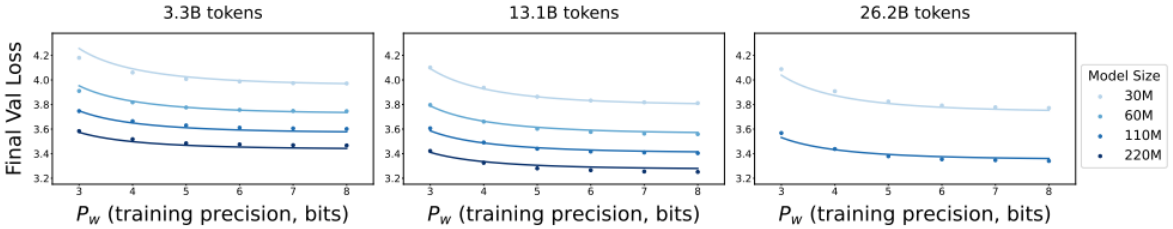
Вывод 2: Эффекты W, A, KV - независимые и мультипликативные

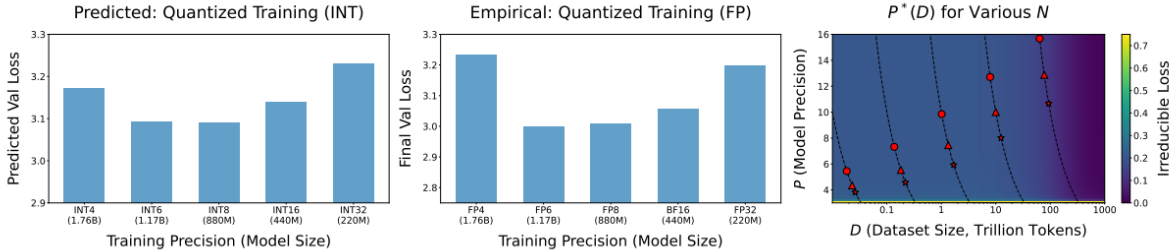
Вывод 3: Оптимальная точность обучения не зависит от бюджета вычислений





Следствия


3.3B tokens
13.1B tokens
26.2B tokens
Final Val Loss
P_w (training precision, bits)
Model Size
30M
60M
110M
220M


Predicted: Quantized Training (INT)
Predicted Val Loss
INT4 (1.76B)
INT6 (1.17B)
INT8 (880M)
INT16 (440M)
INT32 (220M)
Training Precision (Model Size)
Empirical: Quantized Training (FP)
Final Val Loss
FP4 (1.76B)
FP6 (1.17B)
FP8 (880M)
BF16 (440M)
FP32 (220M)
Training Precision (Model Size)
$P^*(D)$ for Various N
P (Model Precision)
D (Dataset Size, Trillion Tokens)
Irreducible Loss

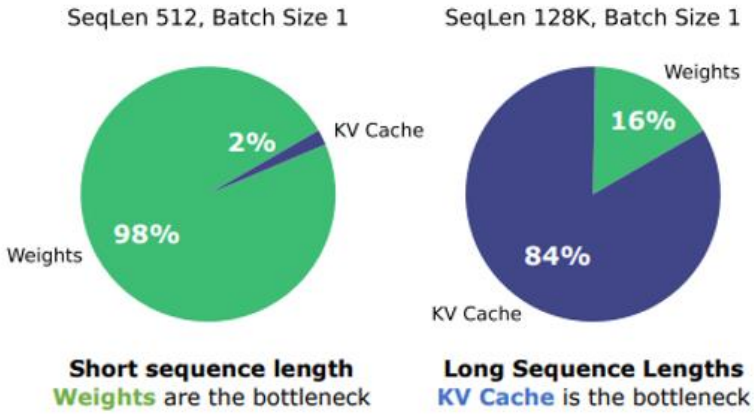
KVQuant

Towards 10 Million Context Length LLM Inference with KV Cache Quantization

2024

При длинном контексте KV-кэш больше самой модели
Цель: сжать KV-кэш до 2 бит

KV-кэш проблема



KVQuant

1.Per-Channel Keys

Выбросы канальные, поэтому своя сетка для каждого канала ключей

2. Pre-RoPE Quantization

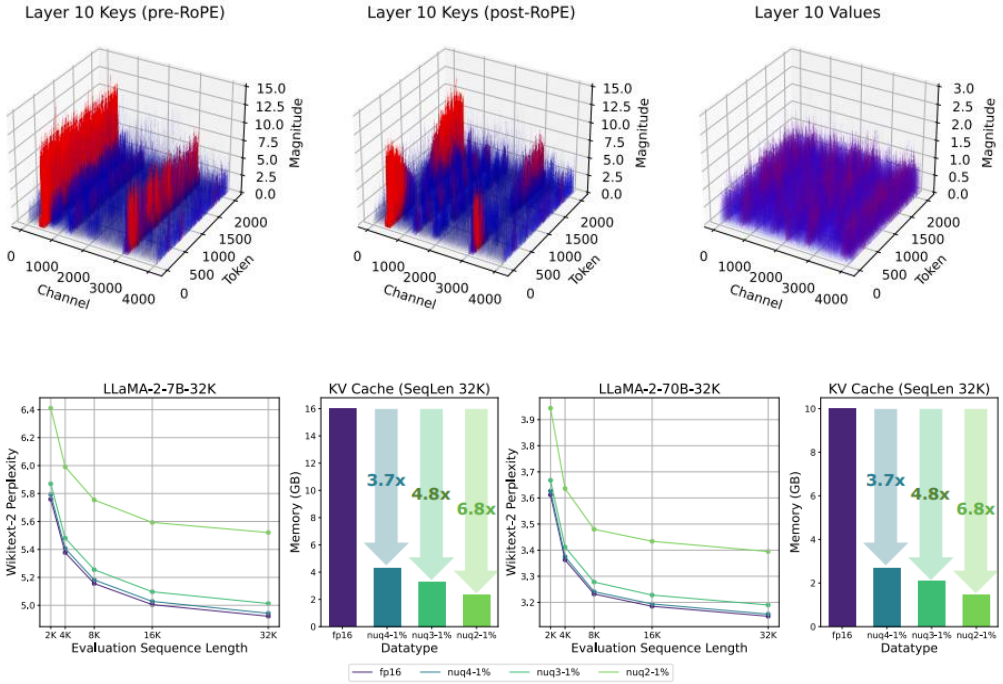
Квантизация ключей до RoPE: значения стабильнее

3.Non-Uniform Quantization

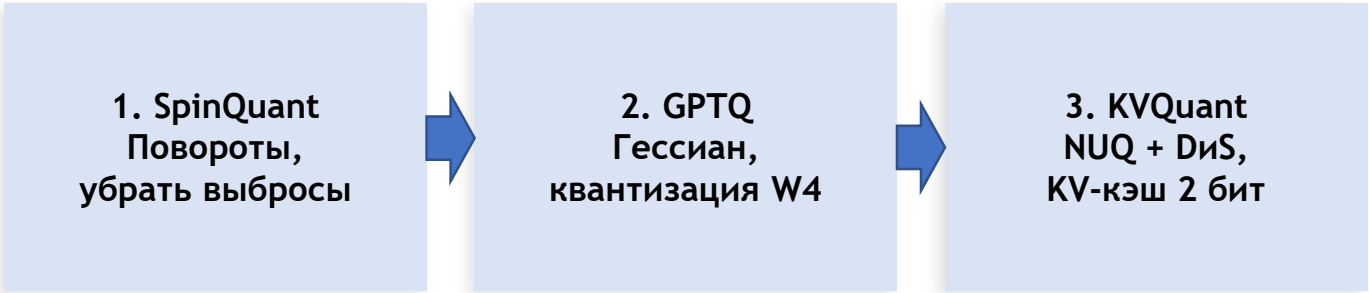
Уровни сетки подстроены под реальное распределение

4. Dense-and-Sparse

Основная масса в 2 бит, выбросы - отдельно в FP16



Общая картина: pipeline квантизации



1. SpinQuant убирает выбросы
2. GPTQ минимизирует ошибку квантизации весов через Гессиан
3. KVQuant сжимает KV-кэш для длинного контекста до 2 бит
4. Scaling Laws дают информацию для выбора конфигурации